

Exercises on cache misses

Exercise 1

A processor has a fully associative L1 cache of 64 B with a block size of 16 B. The replacement policy is RLU. The address and data busses are 32 bits. With respect to the following assembly find:

- a) How many memory blocks are occupied by array?
- b) How many blocks are inside the cache memory?
- c) How many misses occur in the first loop?
- d) How many misses occur in the second loop?
- e) Calculate AMAT assuming a hit time of 1 clock cycle and a miss penalty of 5 clock cycles.

```
.data 0xACDEFF00
array: .space 128

.text

la    $t0, array
li    $t1, 32

loop1:
    lw    $t2, 0($t0)
    addi  $t0, $t0, 4
    addi  $t1, $t1, -1
    bne   $t1, $zero, loop1

la    $t0, array
li    $t1, 32

loop2:
    lw    $t2, 0($t0)
    addi  $t0, $t0, 4
    addi  $t1, $t1, -1
    bne   $t1, $zero, loop2
```

a)
occupied memory blocks -> $128/16 = 8$ blocks

b)
number of blocks $64/16 = 4$ blocks

c)
before loop1
address: 0xACDEFF00
block size 16 B -> 4 bits offset
fully associative cache -> tag = $32 - 4 = 28$ bits offset

Validity bit	Tag	Block
0		
0		
0		
0		

Iteration 1 of loop1

Load the content of 0xACDEFF00 into cache memory (compulsory miss).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
0		
0		
0		

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Iteration 5 of loop1:

Load the content of 0xACDEFF10 into cache memory (compulsory miss).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF1	Word 4 -> 7
0		
0		

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Iteration 9 of loop1

Load the content of 0xACDEFF20 into cache memory (compulsory miss).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF1	Word 4 -> 7
1	0xACDEFF2	Word 8 -> 11
0		

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Iteration 13 of loop1

Load the content of 0xACDEFF30 into cache memory (compulsory miss).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF1	Word 4 -> 7
1	0xACDEFF2	Word 8 -> 11
1	0xACDEFF3	Word 12 -> 15

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Iteration 17 of loop1

Load the content of 0xACDEFF40 into cache memory (compulsory miss).

Since we are using LRU, the first cache block is evicted.

Validity bit	Tag	Block
1	0xACDEFF4	Word 16 -> 19
1	0xACDEFF1	Word 4 -> 7
1	0xACDEFF2	Word 8 -> 11
1	0xACDEFF3	Word 12 -> 15

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Of course, this behaviour is the same for the next iterations and the pattern 1 miss and 3 hit is always the same. At the end of loop1, the cache has the following content.

Validity bit	Tag	Block
1	0xACDEFF4	Word 16 -> 19
1	0xACDEFF5	Word 20 -> 23
1	0xACDEFF6	Word 24 -> 27
1	0xACDEFF7	Word 28 -> 31

For loop1 we have 32 load instructions. We have 8 compulsory misses and 24 hits.

d)

before loop2 the cache stores the same data as at the end of loop1.

Iteration 1 of loop2

Load the content of 0xACDEFF00 into cache memory (capacity miss, since this address has already been cached in the past). Since the replacement policy is LRU, the content of the first cache line is evicted).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF5	Word 20 -> 23
1	0xACDEFF6	Word 24 -> 27
1	0xACDEFF7	Word 28 -> 31

In the next 3 loop2 iterations we have 3 hits (one hit per iteration).

Iteration 5 of loop2:

Load the content of 0xACDEFF10 into cache memory (capacity miss, since this address has already been cached in the past). Since the replacement policy is LRU, the content of the second cache line is evicted).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF1	Word 4 -> 7
1	0xACDEFF6	Word 24 -> 27
1	0xACDEFF7	Word 28 -> 31

As can be easily understood, we have the same access patterns of loop1, but all the misses (8 also in loop2) are capacity misses.

e)

$$AMAT = HitTime + MissRate \times MissPenalty$$

Number of memory accesses = 64 (32 in loop1 and 32 in loop2)

Number of misses = 16 (8 in loop1 and 8 in loop2)

$$MissRate = \frac{16}{64} = 0.25$$

$$AMAT = 1 + 0.25 \times 5 = 2.25$$

Exercise 2

A processor has a 2-way set associative L1 cache of 128 B with a block size of 16 B. The replacement policy is RLU. There is also a victim cache of one block. The address and data busses are 32 bits. With respect to the following assembly find:

- How are the address split?
- How many misses occur?
- Compute the AMAT assuming a hit time of 1 clock cycle and a miss penalty of 10 clock cycles.

```
.data 0x10001000
```

```
A: .space 144
```

```
.text
```

```
la $t0, A
```

```
la $t1, B
```

```
lw $t2, 0($t0)
```

```
lw $t2, 64($t0)
```

```
lw $t2, 128($t0)
```

```
lw $t2, 0($t0)
```

```
lw $t2, 64($t0)
```

```
lw $t2, 128($t0)
```

a)

number of cache blocks = 128/16= 8

number of lines in each way = 8/2 = 4

block size 16 B -> 4 bits offset

number of lines 4 -> index 2 bit

tag = 32 - 4 - 2 =26

Way 0		
Validity bit	Tag	Block
0		
0		
0		
0		

Way 1		
Validity bit	Tag	Block
0		
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
0		

lw \$t2, 0(\$t0) -> the referenced address is 0x10001000

To better split the bits, we use the binary format instead of hexadecimal.

bx00010000000000000001000000000000

The offset is highlighted in red.

The index is highlighted in blue.

The tag is highlighted in green.

We have a compulsory miss.

Way 0		
Validity bit	Tag	Block
1	00010000000000000001000000	Word 0 -> 3
0		
0		
0		

Way 1		
Validity bit	Tag	Block
0		
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
0		

lw \$t2, 64(\$t0) -> the referenced address is 0x10001040

bx00010000000000000001000000000000

We have a compulsory miss. The referenced cache index is again 0, so the block is stored in Way1

Way 0		
V bit	Tag	Block
1	00010000000000000001000000	Word 0 -> 3
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000001	Word 64 -> 67
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
0		

lw \$t2, 128(\$t0) -> the referenced address is 0x10001080

bx00010000000000000001000000000000

We have a compulsory miss. Again, the referenced cache index is 0, the evicted block is the one in Way 0 (LRU). It is copied to the victim cache (with two more bits in the tag!)

Way 0		
V bit	Tag	Block
1	00010000000000000001000010	Word 128 -> 131
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000001	Word 64 -> 67
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
1	0001000000000000000100000000	Word 0 -> 3

lw \$t2, 0(\$t0) -> the referenced address is again 0x10001000

It is stored in the victim cache -> we have a victim hit! The evicted block is the one in Way1 (LRU).

Way 0		
V bit	Tag	Block
1	00010000000000000001000010	Word 128 -> 131
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000000	Word 0 -> 3
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
1	0001000000000000000100000100	Word 64 -> 67

lw \$t2, 64(\$t0) -> the referenced address is again 0x10001040

It is stored in the victim cache -> we have a victim hit! The evicted block is the one in Way0.

Way 0		
V bit	Tag	Block
1	00010000000000000001000001	Word 64 -> 67
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000000	Word 0 -> 3
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
1	0001000000000000000100001000	Word 128 -> 131

lw \$t2, 128(\$t0) -> the referenced address is again 0x10001080

It is stored in the victim cache -> we have a victim hit! The evicted block is the one in Way1.

Way 0		
V bit	Tag	Block
1	00010000000000000001000001	Word 64 -> 67
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000010	Word 128 -> 131
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
1	0001000000000000000100000000	Word 0 -> 3

c)

$$AMAT = HitTime + MissRate \times MissPenalty$$

Number of memory accesses = 6

Number of misses = 3

$$MissRate = \frac{3}{6} = 0.5$$

$$AMAT = 1 + 0.5 \times 10 = 6$$

Exercise 3

A processor has a 2-way set associative L1 cache with a block size of 16 B and 16 lines. The replacement policy is RLU. The processor has a victim cache of 2 lines. The address and data busses are 32 bits. With respect to the following assembly code, compute the real size of the 2-way set associative cache memory and the AMAT (hit time and miss penalty as in Exercise 2).

```
.data 0xFFAB00A0
A: .space 16
.data 0xFFAB01A0
B: .space 16
.data 0xFFAB02A0
C: .space 16
.data 0xFFAB03A0
D: .space 16
```

```
.text
la $t0, A
la $t1, B
la $t2, C
la $t3, D
```

```
lw $s0, 0($t0)
lw $s0, 0($t1)
lw $s0, 0($t2)
lw $s0, 0($t3)
lw $s0, 0($t0)
lw $s0, 0($t1)
lw $s0, 0($t2)
lw $s0, 0($t3)
```

a)

block size 16 B -> 4 bits offset

number of lines 16 -> 4 bits for the cache index

tag = 32-4-4 = 24 bits

The, each way has 1 validity bit, 24 tag bits and 16 B (128 bits) block for each line.

Real cache size for each way = (1+24+128) * 16 = 2448 bits = 306 B

Since the cache has 2 ways the total real size is 612 B.

lw \$s0, 0(\$t0) -> The referenced address is 0xFFAB00A0, the offset is 0x0, the cache line is 0xA and the tag is 0xFFABB00.

The cache is empty, thus we have a compulsory miss.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB00	Word 0->3
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
0		
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
0		
0		

lw \$s0, 0(\$t1) -> the referenced address is 0xFFAB01A0, the offset is 0x0, the cache line is 0xA and the tag is 0xFFAB01

This load causes a compulsory miss. Since the reference cache line is again 0xA, the block is cached in way1.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB00	Word 0->3 of A
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB01	Word 0->3 of B
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
0		
0		

lw \$s0, 0(\$t2) -> the referenced address is 0xFFAB02A0, the offset is 0x0, the cache line is 0xA and the tag is 0xFFAB02

This load causes a compulsory miss. Since the reference cache line is again 0xA, the block is cached in way0.

The evicted block goes in the victim cache.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB02	Word 0->3 of C
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB01	Word 0->3 of B
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB00A	Word 0->3 of A
0		

lw \$s0, 0(\$t3) -> the referenced address is 0xFFAB03A0, the offset is 0x0, the cache line is 0xA and the tag is 0xFFAB03

This load causes a compulsory miss. Since the reference cache line is again 0xA, the block is cached in way1. The evicted block goes in the victim cache.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB02	Word 0->3 of C
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB03	Word 0->3 of D
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB00A	Word 0->3 of A
1	FFAB01A	Word 0->3 of B

lw \$s0, 0(\$t0) -> The referenced address is 0xFFAB00A0, the block is stored in the victim cache. This block is swapped with the one in Way0.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB00	Word 0->3 of A
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB03	Word 0->3 of D
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB02A	Word 0->3 of C
1	FFAB01A	Word 0->3 of B

lw \$s0, 0(\$t1) -> the referenced address is 0xFFAB01A0, the block is stored in the victim cache. This block is swapped with the one in Way1.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB00	Word 0->3 of A
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB01	Word 0->3 of B
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB02A	Word 0->3 of C
1	FFAB03A	Word 0->3 of D

lw \$s0, 0(\$t2) -> the referenced address is 0xFFAB02A0, the block is stored in the victim cache. This block is swapped with the one in Way0.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB02	Word 0->3 of C
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB01	Word 0->3 of B
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB00A	Word 0->3 of A
1	FFAB03A	Word 0->3 of D

lw \$s0, 0(\$t3) -> the referenced address is 0xFFAB03A0, the block is stored in the victim cache. This block is swapped with the one in Way1.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB02	Word 0->3 of C
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB03	Word 0->3 of D
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB00A	Word 0->3 of A
1	FFAB01A	Word 0->3 of B

Number of memory accesses = 8

Number of misses = 4

$$MissRate = \frac{4}{8} = 0.5$$

$$AMAT = 1 + 0.5 \times 10 = 6$$

Exercise 4

A processor has a direct mapped L1 cache with a block size of 16 B and a size of 32 B and a L2 direct mapped cache with a block size of 16 B and a size of 64 B. The address and data busses are 8 bits. With respect to the following assembly code, compute the AMAT (Hit Time L1=1, Hit Time L2=5, Miss Penalty L2 =50).

```
.data 0x00
A: .space 16
.data 0x20
B: .space 16
.data 0x40
C: .space 16
```

```
.text
la $t0, A
la $t1, B
la $t2, C
```

```
lw $s0, 0($t0)
lw $s0, 0($t1)
lw $s0, 0($t0)
lw $s0, 0($t2)
lw $s0, 0($t1)
lw $s0, 0($t0)
```

L1 cache

Block size 16 B -> 4 bit offset

Number of lines -> 32B/16B = 2 -> 1 cache index bit

Tag -> 8-4-1= 3 bits

L2 cache

Block size 16 B -> 4 bit offset

Number of lines -> 64B/16B = 4 -> 2 cache index bits

Tag -> 8-4-2=2 bits

L1			
index	V bit	Tag	Block
0	0		
1	0		

L2			
index	V bit	Tag	Block
0	0		
1	0		
2	0		
3	0		

lw \$s0, 0(\$t0) -> The referenced address is 0x00 -> Compulsory miss and caching in L1 and L2.

Address split in L1 -> bx000 0 0000

Address split in L2 -> bx00 00 0000

L1			
index	V bit	Tag	Block
0	1	000	Array A
1	0		

L2			
index	V bit	Tag	Block
0	1	00	Array A
1	0		
2	0		
3	0		

lw \$s0, 0(\$t1) -> The referenced address is 0x20 -> Compulsory miss and caching in L1 and L2.

Address split in L1 -> bx001 0 0000

Address split in L2 -> bx00 10 0000

L1			
index	V bit	Tag	Block
0	1	001	Array B
1	0		

L2			
index	V bit	Tag	Block
0	1	00	Array A
1	0		
2	1	00	Array B
3	0		

lw \$s0, 0(\$t0) -> The referenced address is 0x00 -> miss in L1 and hit L2.

Address split in L1 -> bx000 0 0000

Address split in L2 -> bx00 00 0000

L1			
index	V bit	Tag	Block
0	1	000	Array A
1	0		

L2			
index	V bit	Tag	Block
0	1	00	Array A
1	0		
2	1	00	Array B
3	0		

lw \$s0, 0(\$t2) -> The referenced address is 0x40 -> Compulsory miss and caching in L1 and L2.

Address split in L1 -> bx010 0 0000

Address split in L2 -> bx01 00 0000

L1			
index	V bit	Tag	Block
0	1	010	Array C
1	0		

L2			
index	V bit	Tag	Block
0	1	01	Array C
1	0		
2	1	00	Array B
3	0		

lw \$s0, 0(\$t1) The referenced address is 0x20 -> miss in L1 and hit in L2

Address split in L1 -> bx001 0 0000

Address split in L2 -> bx00 10 0000

L1			
index	V bit	Tag	Block
0	1	001	Array B
1	0		

L2			
index	V bit	Tag	Block
0	1	01	Array C
1	0		
2	1	00	Array B
3	0		

lw \$s0, 0(\$t0) The referenced address is 0x00 -> miss in L1 and L2 and caching.

Address split in L1 -> bx000 0 0000

Address split in L2 -> bx00 00 0000

L1			
index	V bit	Tag	Block
0	1	000	Array A
1	0		

L2			
index	V bit	Tag	Block
0	1	00	Array A
1	0		
2	1	00	Array B
3	0		

To summarize:

Instruction	L1	L2
lw \$s0, 0(\$t0)	Miss	Miss
lw \$s0, 0(\$t1)	Miss	Miss
lw \$s0, 0(\$t0)	Miss	Hit
lw \$s0, 0(\$t2)	Miss	Miss
lw \$s0, 0(\$t1)	Miss	Hit
lw \$s0, 0(\$t0)	Miss	Miss

Number of memory accesses = 6

Number of misses in L1 = 6

Number of misses in L2 = 4

$$MR_{L1} = \frac{6}{6} = 1$$

$$MR_{L2} = \frac{4}{6} = 0.\bar{6}$$

$$AMAT = HT_{L1} + MR_{L1} \times (HT_{L2} + MR_{L2} \times MP_{L2}) = 1 + 1 \times (5 + 0.\bar{6} \times 50) = 39.\bar{3}$$